SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE NAME: INTERNET PROGRAMMING **COURSE CODE: CSA4305**

COURSE OUTCOME COVERED

CO-1: Develop well-structured, standards-compliant web pages using HTML/XHTML and XML, and apply Internet protocols and HTTP communication mechanisms for web content delivery. (L2)

Assessment Tool: Application-Based Questions
Weightage: 40%

QUESTIONS: Total Marks: 30

1: Online Symposium Portal

A college is developing an online symposium registration portal. The following HTML structure and HTTP interaction is observed:

```
<form action="/register" method="POST">
  <input type="text" name="name">
  <input type="email" name="email">
  <button>Submit</button>
</form>
```

When a student submits the form, the browser sends an HTTP request to the server, and the server responds accordingly.

Questions:

1. Identify appropriate **HTML5 semantic elements** missing in the structure.

Answer: The missing HTML5 semantic elements that should be used to wrap and structure the form components include:

- <section>: To group the form content as a standalone registration component.

- <fieldset>: To logically group the input controls (Name, Email) inside the form.

- <legend>: To provide a descriptive caption for the grouped input controls.

- <label>: To explicitly associate descriptions with the input fields (essential for accessibility).

2. Modify the structure to make it **standards-compliant**.

Answer: `The modified, standards-compliant, and accessible HTML5 structure is shown below:`

```
<section>
  <form action="/register" method="POST">
    <fieldset>
      <legend>Symposium Registration</legend>
      <p>
        <label for="student-name">Full Name:</label>
        <input type="text" id="student-name" name="name" required
placeholder="Enter your name">
      </p>
      <p>
        <label for="student-email">Email Address:</label>
        <input type="email" id="student-email" name="email" required
placeholder="Enter your email">
      </p>
      <button type="submit">Submit Registration</button>
    </fieldset>
  </form>
</section>
```

3. Interpret the **HTTP request generated** during form submission.

Answer: Since the form uses method="POST", submitting it causes the browser to send a POST request with the form data in the body:

```
POST /register HTTP/1.1
Host: symposium.college.edu
Content-Type: application/x-www-form-urlencoded
Content-Length: 42

name=John+Doe&email=johndoe%40college.edu
```

4. Analyze the **HTTP response cycle** from server to browser.

Answer: The HTTP response cycle consists of the following phases:

1. Request Dispatch: Browser establishes a TCP connection (typically port 443/HTTPS) and transmits the POST request.

2. Server Processing: The web server matches the route (/register), extracts parameters from the request body, executes business logic (e.g., saving to database), and generates the response content.

3. Response Return: The server returns a status code (e.g., HTTP/1.1 200 OK or HTTP/1.1 303 See Other redirecting to a success page) along with headers and body.

4. Client Rendering: The browser receives the response, parses the payload, updates the DOM, and displays the registration status page to the student.

5. Suggest improvements for **usability and accessibility**.

Answer: Usability and accessibility can be improved with the following enhancements:

- Label Association: Explicitly connect labels and inputs using the 'for' and 'id' attributes so screen readers function correctly.

- Client-side Validation: Apply 'required' attributes to inputs to catch missing fields before request submission.

- Usability Cues: Use placeholders and appropriate keyboard/focus visual styles (active borders) to guide users.

- Button Type Declaration: Define type="submit" to make form submission explicit.

2: Cross-Browser Compatibility Issue

A company website shows inconsistent layout across browsers. The following HTML snippet is used:

```
<html>
  <head>
    <title>Company</title>
  </head>
  <body>
    <div>
      <h1>Welcome
      <p>About us
    </div>
  </body>
</html>
```

Questions:

1. Identify **improper nesting and missing closing tags**.

Answer: The syntax issues in the provided snippet are:

- The <h1> tag is never closed, leading to improper nesting with the subsequent <p> tag.

- The <p> tag is never closed, which means it relies on browser-specific auto-closing logic when meeting .

2. Analyze how **different browsers may interpret this structure**.

Answer: Modern browsers construct the DOM using the HTML5 specification parser rules, but differences still arise:

- Chrome/Firefox/Safari: Since <h1> and <p> are block-level elements, the parser auto-closes the open <h1> when it encounters the starting <p> tag. It then closes the <p> tag upon encountering the closing tag.

- Older browsers / Quirks Mode: Some parsers may nest the <p> element inside the <h1> tag, applying heading formatting (large, bold text) to the "About us" paragraph, disrupting the designed visual structure.

3. Identify **missing semantic elements** (e.g., header, section).

Answer: The missing elements are:

- <!DOCTYPE html>: Standard declaration that triggers HTML5 standards mode and avoids quirks mode.

- <meta charset="UTF-8">: Standard character encoding.

- <header>: Ideal container for wrapping the heading (<h1>).

- <main> or <section>: More semantic than a generic for grouping body sections.

4. Provide a **corrected W3C-compliant HTML structure**.

Answer: The corrected W3C-compliant structure is:

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Company</title>
</head>
<body>
  <header>
    <h1>Welcome</h1>
  </header>
  <main>
    <section>
      <p>About us</p>
    </section>
  </main>
</body>
</html>
```

5. Suggest techniques to ensure **cross-browser compatibility**.

Answer: The following techniques ensure cross-browser layout consistency:

- CSS Reset or Normalization: Include Normalize.css or a custom CSS reset to eliminate default stylesheet discrepancies across engines.

- W3C Validation: Test markup with validation tools (like validator.w3.org) to check for parsing issues.

- Mandatory DOCTYPE: Always include <!DOCTYPE html> at the first line.

- CSS Feature Detection: Use CSS @supports rule or Modernizr JS to detect capabilities and offer clean fallback styles.

3: Form Submission Failure

A web application fails to send form data to the server. The following configuration is observed:

```
<form action="/submit" method="GET">
  <input type="text" name="username">
  <input type="password" name="password">
  <button type="button">Login</button>
</form>
```

Questions:

1. Identify issues in the **form configuration**.

Answer: The configured form suffers from two primary issues:

- Button type is button: The submit button uses type="button", which disables standard submission behavior.

- Method is GET: Transmitting credentials (password) using GET query strings represents a significant security hazard.

2. Analyze why the **HTTP request is not triggered**.

Answer: The HTTP request is not triggered because:

- In HTML5, buttons inside a form must be of type "submit" to trigger submission. Setting type="button" creates a generic button that does not submit the form unless handled by custom JavaScript. Since no JS event handler is specified, clicking does nothing.

3. Interpret the role of **GET method in this scenario**.

Answer: The GET method appends the parameters to the request URL as query parameters:

```
Example URL: /submit?username=myuser&password=mypassword
This is insecure because passwords will be saved in plain text in browser
histories, bookmarks, and server access/proxy logs.
```

4. Propose a **corrected implementation**.

Answer: `The secure, corrected form structure is:`

```
<form action="/submit" method="POST">
  <p>
    <label for="usr">Username:</label>
    <input type="text" id="usr" name="username" required>
  </p>
  <p>
    <label for="pwd">Password:</label>
    <input type="password" id="pwd" name="password" required>
  </p>
```

```
  <button type="submit">Login</button>
</form>
```

5. Suggest improvements for **secure data transmission**.

Answer: To ensure secure transmission, implement the following standard practices:

- Change GET to POST: Ensure passwords and user tokens are submitted in the HTTP request body.

- Force HTTPS (SSL/TLS): Always load the portal over TLS/SSL so data is encrypted in transit and protected from interceptors.

- CSRF Defense: Include an anti-CSRF token in the form.

- Strict Validation: Enforce input sanitation and rate-limiting on login endpoints.

Code of Conduct:

I _____Enamala Gowtham (192311190) _____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: Enamala Gowtham

RUBRICS FOR CALCULATION:

Criteria	Weigh htage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Concepts	25%	Demonstrates thorough, accurate understanding of concepts	Good understanding with minor gaps	Basic understanding with some misconceptions	Poor understanding; major misconceptions
Explanation	25%	Detailed, well explained answers with relevant examples	Clear explanation with adequate details	Limited explanation; lacks depth	Poor or unclear explanation
Application	20%	Effectively applies concepts with strong analytical ability	Applies concepts with minor errors	Limited application with weak analysis	No application or incorrect analysis
Organization	15%	Well-structured answers with logical flow and	Organized with minor issues in flow	Basic structure, lacks clarity	Poor organization, incoherent

		coherence			answers
Accuracy	10%	Completely accurate and covers all required points	Mostly accurate with minor omissions	Partially correct with missing points	Incorrect answers with major gaps
Clarity	5%	Clear, neat, professional presentation	Understandable with minor issues	Limited clarity, readability issues	Poorly written, difficult to understand